components of logical expressions are the *logical* operators, *comparison* operators, and *bitwise* operators. These are the units of code that determine the correct functioning of your program.

What Is Truth?

Program logic is driven by the idea of things being true or false. Code branches, and the flow of execution goes one way or another, based on decisions made on the basis of an expression being true or false. But here's the thing: how are these two absolute states represented? Well, we have the `bool` data type, which has two possible values of `true` or `false`. That's nice, but this is a loosely-typed language. Strings can become numbers, and objects can become strings, often without any intervention on the programmer's part. `bool` is no different: if an expression is evaluated in a boolean context, type coercion will happen. Like other loosely typed languages, there is a concept of "truthy" and "falsey": non-boolean values that are equivalent to `true` or `false`. Here's a synopsis:

```
var_dump((bool) null); // bool(false)
var_dump((bool) 0); // bool(false)
var_dump((bool) 0.0); // bool(false)
var_dump((bool) []); // bool(false)
var_dump((bool) ['foo']); // bool(true)
var_dump((bool) ''); // bool(false)
var_dump((bool) 'literally anything'); // bool(true)
var_dump((bool) '0'); // bool(false)!
```

As you can see, there's a gotcha: the string `'0'` is not empty; it contains a byte value of *0x30*, which has the binary value of `00110000`, yet it's false. I highly recommend being as type-safe as possible when it comes to logical expressions (and, to be fair, all of your programming).

Control Structures

Control structures are simply a way to manage the order of execution of statements. By testing for a condition, it is possible to skip a set of statements or ensure statements that would otherwise be skipped are executed. They can be broadly grouped into two types: decision-making and repetition.